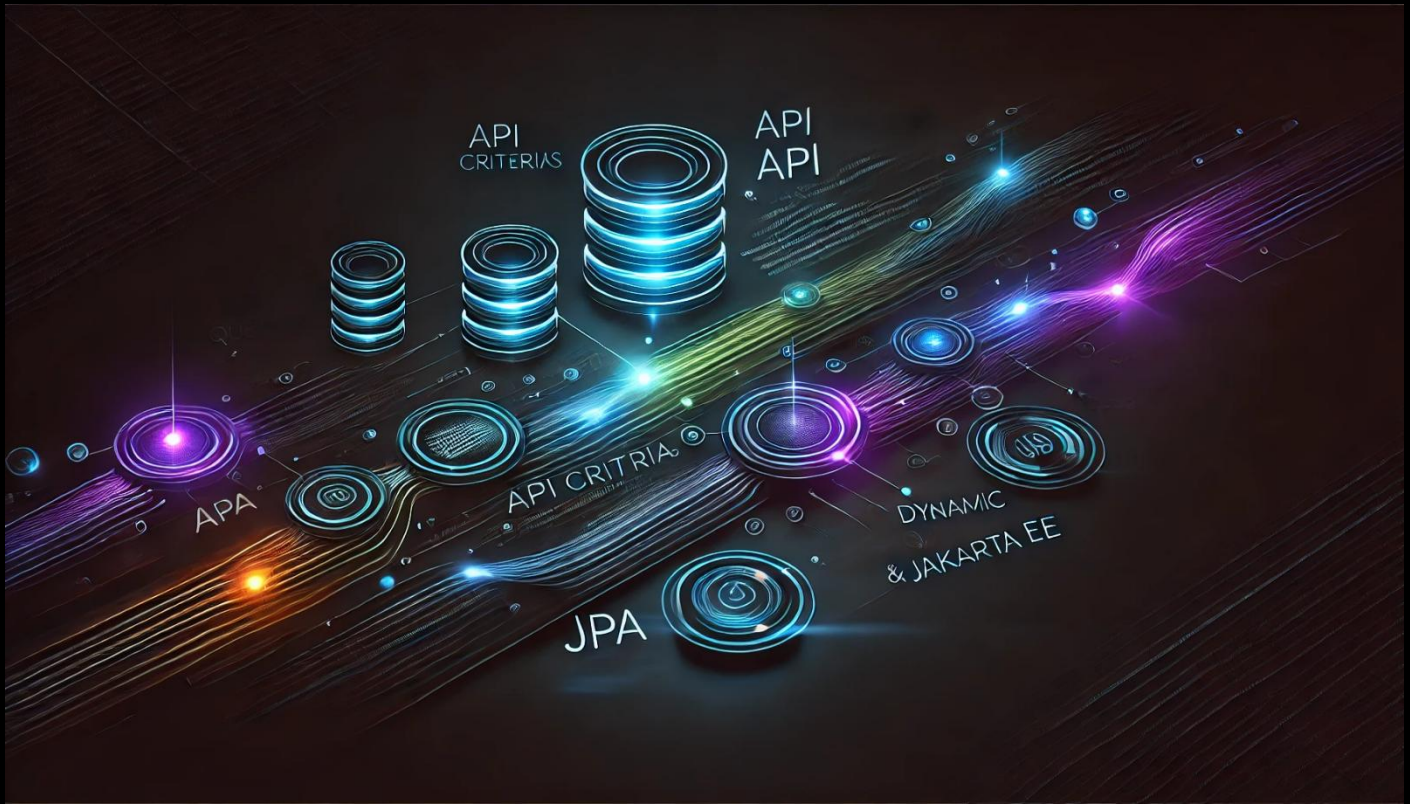


API de Criteria con JPA – parte 2



Consulta con API de Criteria y Parámetros Dinámicos en JPA 🚀

En esta guía aprenderás cómo **filtrar registros en JPA usando la API de Criteria** y cómo aplicar **parámetros dinámicos** para consultas flexibles. Veremos dos enfoques para realizar una consulta filtrando por **idPersona**, y cómo mejorar la consulta con **Predicate** para agregar múltiples filtros de manera dinámica.

1 Conceptos Básicos de la API de Criteria 📁

♦ ¿Qué es la API de Criteria?

La **API de Criteria** permite construir consultas programáticas en Java sin necesidad de escribir consultas JPQL como cadenas de texto. Esto ofrece **tipado seguro, mayor flexibilidad y la posibilidad de construir consultas dinámicamente**.

♦ Ventajas de Criteria:

- ✓ **Evita errores de sintaxis en consultas** al no depender de cadenas de texto.
- ✓ **Permite construir consultas dinámicas** con filtros opcionales.
- ✓ **Manejo más seguro de parámetros** con `Predicate`.

◆ Sintaxis Básica para una Consulta con Criteria:

```
// 1. Obtener el CriteriaBuilder desde el EntityManager
CriteriaBuilder cb = entityManager.getCriteriaBuilder();

// 2. Crear una consulta de Criteria para una entidad específica
CriteriaQuery<Persona> criteriaQuery = cb.createQuery(Persona.class);

// 3. Definir la raíz de la consulta
Root<Persona> fromPersona = criteriaQuery.from(Persona.class);

// 4. Agregar filtros con Predicate
criteriaQuery.select(fromPersona).where(cb.equal(fromPersona.get("idPersona"), 1));

// 5. Ejecutar la consulta
TypedQuery<Persona> query = entityManager.createQuery(criteriaQuery);
Persona persona = query.getSingleResult();
```

2 Implementación Paso a Paso

Mejoras en el Código

- ◆ **Vamos a agregar dos enfoques diferentes para filtrar por `idPersona`.**
 - ◆ **Uso de `Predicate` para agregar criterios dinámicos de manera flexible.**
-

3 Implementación en Código

```
package sga.cliente.criterias;

import java.util.List;
import jakarta.persistence.*;
import jakarta.persistence.criteria.*;
import java.util.ArrayList;
import sga.domain.Persona;
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;

public class PruebaApiCriteria {

    private static final Logger log = LoggerFactory.getLogger(PruebaApiCriteria.class);

    public static void main(String[] args) {
        // 1. Crear la fábrica de entidades
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("SgaPU");

        try (EntityManager em = emf.createEntityManager()) {

            // 2. Crear el CriteriaBuilder
            CriteriaBuilder cb = em.getCriteriaBuilder();

            // 3. Crear una consulta de Criteria para la entidad Persona
            CriteriaQuery<Persona> criteriaQuery = cb.createQuery(Persona.class);

            // 4. Definir la raíz de la consulta (especificar de qué entidad obtener los datos)
            Root<Persona> fromPersona = criteriaQuery.from(Persona.class);

            // 5. Seleccionar los datos desde la raíz
            criteriaQuery.select(fromPersona);

            // 6. Crear y ejecutar la consulta
            TypedQuery<Persona> query = em.createQuery(criteriaQuery);
            List<Persona> personas = query.getResultList();

            // 7. Mostrar los resultados
            mostrarPersonas(personas);

            // **2-a. Consulta de la Persona con id = 1 (Forma Directa)**
            log.info("\n2-a. Consulta de la Persona con id = 1");
            cb = em.getCriteriaBuilder();
            criteriaQuery = cb.createQuery(Persona.class);
            fromPersona = criteriaQuery.from(Persona.class);
            criteriaQuery.select(fromPersona).where(cb.equal(fromPersona.get("idPersona"), 1));
            Persona persona = em.createQuery(criteriaQuery).getSingleResult();
            log.info("Persona: " + persona);

            // **2-b. Consulta de la Persona con id = 1 (Usando Predicate)**
            log.info("\n2-b. Consulta de la Persona con id = 1 usando Predicate");
            cb = em.getCriteriaBuilder();
            criteriaQuery = cb.createQuery(Persona.class);
            fromPersona = criteriaQuery.from(Persona.class);
```

```

criteriaQuery.select(fromPersona);

// La clase Predicate permite agregar varios criterios dinámicamente
List<Predicate> criterios = new ArrayList<>();

// Verificamos si tenemos criterios que agregar
Integer idPersonaParam = 1;
ParameterExpression<Integer> parameter = cb.parameter(Integer.class, "idPersona");
criterios.add(cb.equal(fromPersona.get("idPersona"), parameter));

// Se agregan los criterios a la consulta
if (criterios.isEmpty()) {
    throw new RuntimeException("Sin criterios");
} else if (criterios.size() == 1) {
    criteriaQuery.where(criterios.get(0));
} else {
    criteriaQuery.where(cb.and(criterios.toArray(Predicate[]::new)));
}

query = em.createQuery(criteriaQuery);
query.setParameter("idPersona", idPersonaParam);

// Se ejecuta el query
persona = query.getSingleResult();
log.info("Persona: " + persona);
}
}

// Método para mostrar los resultados obtenidos
private static void mostrarPersonas(List<Persona> personas) {
    for (Persona p : personas) {
        log.info("Persona: " + p);
    }
}
}
}

```

4 Explicación del Código 🤖

1. Crear la fábrica de entidades

```
EntityManagerFactory emf = Persistence.createEntityManagerFactory("SgaPU");
```

✚ Permite acceder a la base de datos y gestionar las entidades.

2. Crear el CriteriaBuilder

```
CriteriaBuilder cb = em.getCriteriaBuilder();
```

✚ `CriteriaBuilder` se usa para construir consultas programáticas con `Criteria`.

3. Crear una consulta de Criteria

```
CriteriaQuery<Persona> criteriaQuery = cb.createQuery(Persona.class);
```

✚ Se crea un objeto `CriteriaQuery` para trabajar con la entidad `Persona`.

4. Definir la raíz de la consulta

```
Root<Persona> fromPersona = criteriaQuery.from(Persona.class);
```

✚ Define desde qué entidad se extraerán los datos (`FROM Persona` en SQL).

5. Agregar un filtro simple con `where`

```
criteriaQuery.select(fromPersona).where(cb.equal(fromPersona.get("idPersona"), 1));
```

✚ Filtro para recuperar solo el registro con `idPersona = 1`.

6. Uso de `Predicate` para múltiples criterios dinámicos

```
List<Predicate> criterios = new ArrayList<>();
ParameterExpression<Integer> parameter = cb.parameter(Integer.class, "idPersona");
criterios.add(cb.equal(fromPersona.get("idPersona"), parameter));
```

```
if (criterios.isEmpty()) {
    throw new RuntimeException("Sin criterios");
} else if (criterios.size() == 1) {
    criteriaQuery.where(criterios.get(0));
} else {
    criteriaQuery.where(cb.and(criterios.toArray(Predicate[]::new)));
}
```

✚ Se usa `Predicate` para agregar filtros dinámicamente.



Veamos la explicación detallada de la siguiente línea:

```
criteriaQuery.where(cb.and(criterios.toArray(Predicate[]::new)));
```

Esta línea de código está aplicando una **cláusula WHERE** con múltiples condiciones dinámicas en la consulta Criteria de JPA. Para entender mejor su funcionamiento, desglosémosla paso a paso:

◆ 1. ¿Qué es `criteriaQuery.where()` ?

✚ `where()` es un método de `CriteriaQuery` que se utiliza para establecer la condición de filtro (**WHERE**) de la consulta.

Ejemplo básico:

```
criteriaQuery.where(cb.equal(fromPersona.get("idPersona"), 1));
```

- ◆ En este caso, `where()` filtra únicamente las filas donde `idPersona` sea igual a 1.
-

◆ 2. ¿Qué hace `cb.and(...)` ?

✚ `cb.and(...)` es un método de `CriteriaBuilder` que permite **combinar múltiples condiciones** con el operador lógico **AND**.

Ejemplo:

```
criteriaQuery.where(cb.and(  
    cb.equal(fromPersona.get("nombre"), "Juan"),  
    cb.equal(fromPersona.get("apellido"), "Pérez")  
));
```

- ◆ En este caso, el **AND** filtra únicamente las filas donde el `nombre` sea "Juan" y el `apellido` sea "Pérez".
-

◆ 3. ¿Qué es `criterios.toArray(Predicate[]::new)` ?

✚ `criterios` es una lista (`List<Predicate>`) que almacena dinámicamente los filtros de la consulta.

Ejemplo:

```
List<Predicate> criterios = new ArrayList<>();
```

```
criterios.add(cb.equal(fromPersona.get("nombre"), "Juan"));
criterios.add(cb.equal(fromPersona.get("apellido"), "Pérez"));
```

♦ Esta lista almacena **una cantidad variable** de filtros. Ahora bien, `cb.and(...)` requiere recibir **un array de Predicate**, por lo que debemos convertir la lista en un array.

✅ `criterios.toArray(Predicate[]::new)` convierte la lista de `Predicate` en un array dinámico de `Predicate[]` para pasarlo a `cb.and(...)`.

Explicación Completa de la Línea

```
criteriaQuery.where(cb.and(criterios.toArray(Predicate[]::new)));
```

Desglose de cada parte:

1. `criterios.toArray(Predicate[]::new)`
 - ♦ Convierte la lista `criterios` en un array de `Predicate`.
2. `cb.and(...)`
 - ♦ Une todas las condiciones de `criterios` con AND.
3. `criteriaQuery.where(...)`
 - ♦ Aplica los filtros a la consulta.

✅ Ejemplo con múltiples filtros dinámicos:

```
List<Predicate> criterios = new ArrayList<>();

// Agregamos condiciones dinámicamente
criterios.add(cb.equal(fromPersona.get("nombre"), "Juan"));
criterios.add(cb.like(fromPersona.get("apellido"), "%P%"));
criterios.add(cb.greaterThan(fromPersona.get("idPersona"), 10));

// Aplicamos la condición WHERE con AND
criteriaQuery.where(cb.and(criterios.toArray(Predicate[]::new)));
```

♦ Esto generará una consulta equivalente a:

```
SELECT * FROM Persona
WHERE nombre = 'Juan'
AND apellido LIKE '%P%'
```

```
AND idPersona > 10;
```

Conclusión del análisis anterior

- ✅ `criteriaQuery.where(cb.and(criterios.toArray(Predicate[]::new)))`; permite **agregar múltiples condiciones dinámicamente**.
- ✅ Se usa `cb.and(...)` para **combinar todas las condiciones con AND**.
- ✅ `criterios.toArray(Predicate[]::new)` **convierte la lista de filtros en un array** para que pueda ser usado por `cb.and(...)`.

Así, podemos agregar filtros condicionalmente y construir **consultas flexibles y dinámicas** con Criteria en JPA. 🚀

5 Salida Esperada en la Consola

Si la base de datos tiene registros en la tabla `Persona`, verás un resultado similar a este:

```
2-a. Consulta de la Persona con id = 1
[EL Fine]: sql: ServerSession(1298146757)--Connection(1818747191)--SELECT id_persona, APELLIDO, EMAIL,
NOMBRE, TELEFONO FROM PERSONA WHERE (id_persona = ?)
      bind => [1]
[main] INFO sga.cliente.criteria.PruebaApiCriteria - Persona: Persona{idPersona=1, nombre=Rafael,
apellido=Juarez, email=rjuarez@mail.com, telefono=55667788}
[main] INFO sga.cliente.criteria.PruebaApiCriteria -

2-b. Consulta de la Persona con id = 1 usando Predicate
[EL Fine]: sql: ServerSession(1298146757)--Connection(1818747191)--SELECT id_persona, APELLIDO, EMAIL,
NOMBRE, TELEFONO FROM PERSONA WHERE (id_persona = ?)
      bind => [1]
[main] INFO sga.cliente.criteria.PruebaApiCriteria - Persona: Persona{idPersona=1, nombre=Rafael,
apellido=Juarez, email=rjuarez@mail.com, telefono=55667788}
```

6 Conclusión

- ✅ Aprendimos cómo filtrar registros con Criteria en JPA.
 - ✅ Vimos dos enfoques: consulta directa y usando `Predicate`.
-

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)